Exploration of GPU-enabled lossless compressors

Stefan Rua, stefan.elias.rua@cern.ch

Contents

Introduction **1**

Algorithms **2**
Burrows-Wheeler transform . . . 2
Huffman coding . . . 2
The Lempel-Ziv family . . . 2
Asymmetric numeral systems . . . 2

CPU compressors **2**
Zstandard . . . 2
xz . . . 2

GPU compressors **2**
BSC . . . 2
CULZSS . . . 2
nvCOMP . . . 3
DietGPU . . . 3
HuffmanCoding_MPI_CUDA . . . 3
cuda_bzip2 . . . 3

Method **3**
Crude first tests . . . 3
Finer evaluation . . . 3

Results **4**

Discussion **5**

Conclusion **5**

Introduction

The data produced by CMS's HLT is compressed, and currently this is done using CPUs. However, as new computers with powerful GPUs have been acquired, it would be nice to transfer some of the load onto the GPUs. This is the motivation for finding a good GPU-enabled compressor.

GPU-enabled compressors aren't widely adopted yet (?), but in the last ten years some researchers and companies have showed interest in the topic.

Algorithms

Burrows-Wheeler transform

Huffman coding

The Lempel-Ziv family

Asymmetric numeral systems

CPU compressors

Zstandard

Zstandard (zstd) is the compressor that is currently used for HLT data. It is used as a reference point for all comparisons in this report.

xz

Another point of comparison running on the CPU is xz, which achieves great compression ratios at the cost of throughput.

GPU compressors

BSC

BSC[1] is a GPU accelerated block-sorting compressor. The default makefile doesn't compile the necessary files for CUDA support, so a few changes were needed (?). Multiple block sorting algorithms are available: the Burrows-Wheeler transform and sort transforms of order 3 to 8 (?). Only sort transforms of order 5 to 8 can use the GPU, of which 7 and 8 use it by default and 5 and 6 require and additional flag for it.

CULZSS

CULZSS[2] is a CUDA implementation of LZSS. CULZSS-bit[3], a more recent and reportedly better performing version exists, but no source code or binary is available for it. I emailed the author, but he couldn't find the code either. He also mentioned that the code for the original CULZSS on github might not work.

[1] https://github.com/IlyaGrebnov/libbsc

[2] https://github.com/adnanozsoy/CUDA_Compression, https://web.cs.hacettepe.edu.tr/~aozsoy/papers/2011-ppac.pdf

[3] https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7079027

nvCOMP

nvCOMP[4] is a library for GPU compression by Nvidia, and it implements Deflate, LZ4, Cascaded, Snappy, BitComp, and ANS. I expect nvCOMP's compressors to perform well, as it is in Nvidia's interest to make their GPUs look good. Unfortunately nvCOMP is proprietary.

DietGPU

DietGPU[5] is a work-in-progress ("very early alpha preview") compression library by Facebook which implements ANS. As its purpose they state speeding up network transfers in a datacenter environment, which matches our use case. DietGPU provides two codecs: a general byte-oriented entropy encoder and decoder, and another one specifically for floating point data. At the time of writing only a Python API for PyTorch tensors is ready, but the underlying encoders and decoders are implemented in C++ and CUDA, making it possible to utilize library for other data as well.

HuffmanCoding_MPI_CUDA

HuffmanCoding_MPI_CUDA[6] performs Huffman compression with the option of using CUDA, MPI, CUDA and MPI, or no parallelization at all. I used it on the CUDA setting.

cuda_bzip2

cuda_bzip2[7] is a modified version of bzip2 with CUDA support.

Method

Crude first tests

As a first step I ran the compressors from the command line, measuring the time taken by them using the `time` command. This gave me a rough idea of their performance, based on which I selected the ones to examine in more detail.

Finer evaluation

lzbench[8] is an in-memory benchmarking tool for compressors. Excluding the time taken read and write files from the disk gives a more accurate idea of the compressor's throughput. I added the most promising compressors to lzbench.

I failed to add nvCOMP to lzbench, so its results are the ones reported by Nvidia's own proprietary benchmarking binaries. I expect these to exclude all possible overhead, such as copying the files to GPU memory from RAM.

[4] https://developer.nvidia.com/nvcomp
[5] https://github.com/facebookresearch/dietgpu
[6] https://github.com/smadhiv/HuffmanCoding_MPI_CUDA
[7] https://github.com/aditya12agd5/cuda_bzip2
[8] https://github.com/inikep/lzbench

Results

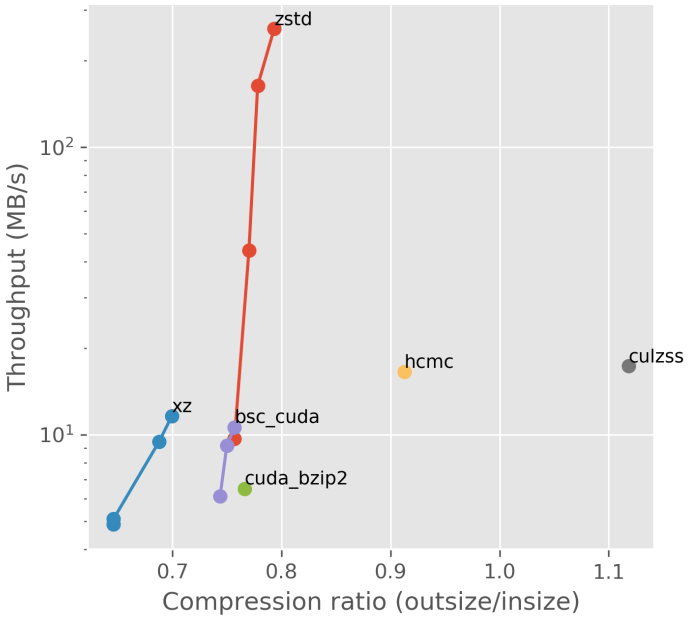

Figure 1: Crude first tests from the command line

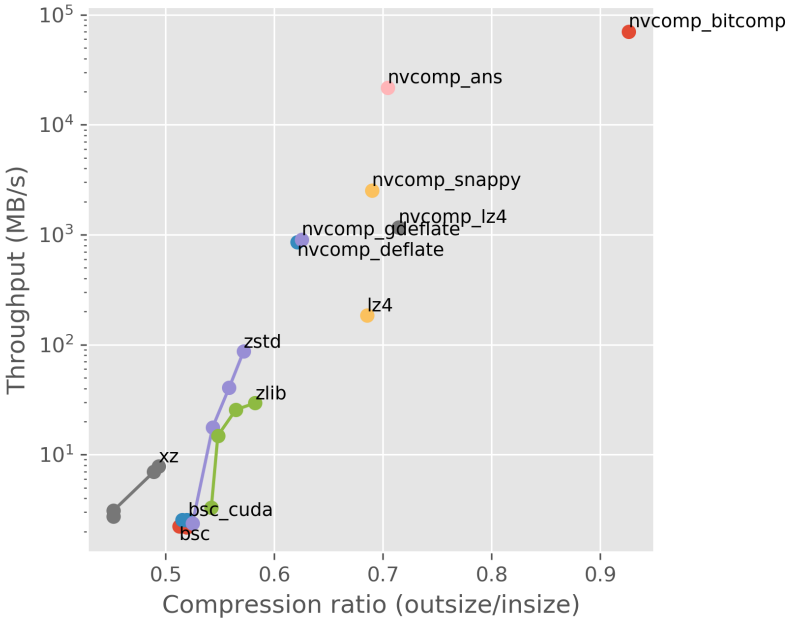

Figure 2: Finer evaluation using lzbench

Discussion

Conclusion